

Паттерн (шаблон)

Паттерны (шаблоны) проектирования — это способы построения программ, которые считаются хорошим тоном для разработчиков. Их еще называют шаблонами или образцами: чаще всего паттерн — это типовое решение для часто встречающейся задачи на построение.

Паттерны используются именно для проектирования и структуризации, а не для бизнес-задач. Можно привести аналогию: если вы разрабатываете самолет, вам не нужно с нуля придумывать, какой формы он будет. Опыт предыдущих самолетостроителей подсказывает правила: формат и количество крыльев, особенности хвоста, нужные элементы. Это и есть паттерн — схема, по которой строится решение.

Понятие не стоит путать с паттерном в дизайне — там так называются узоры, построенные на повторяющихся элементах или орнаментах. В разработке же паттерн — это схема решения.

Кроме паттернов, существуют антипаттерны проектирования — это плохие решения, неэффективные. Применять их считается дурным тоном.

Кто пользуется паттернами проектирования

Паттерны используют программисты на разных языках, в первую очередь те, которые занимаются объектно-ориентированным программированием. Само понятие паттерна тесно связано с ООП: этот подход позволяет разбить структуру программы на формализованные классы и объекты. Решение строится из них, как из кирпичиков. Такой вид разработки хорошо сочетается с паттернами.

В широком смысле паттерны проектирования применяются во всех отраслях, где есть списки часто встречающихся формализованных проблем. В разработке это почти любое направление.

Для чего нужны паттерны

В программировании есть задачи, которые встречаются часто и с которыми сталкивается большинство разработчиков.

Придумывать новое уникальное решение каждый раз было бы трудозатратно и неэффективно, поэтому существуют паттерны, на которые можно ориентироваться.

Паттерны проектирования помогают быстрее и эффективнее создавать код, а не «изобретать велосипеды». Как с созданием

любого продукта: лучше воспользоваться знаниями, которые наработали другие, чем продумывать с нуля абсолютно все.

Если разработчик может грамотно формализовать проблему с помощью ООП и выбрать подходящий паттерн для ее решения, это может серьезно ускорить сроки разработки. А решение будет понятным и эффективным – это уже доказали люди, которые начали применять конкретный паттерн раньше.

Кроме того, использование паттернов еще и улучшает читаемость кода. Другой программист, знакомый с нужным паттерном, сможет увидеть его в коде и понять, как все реализовано. А ведь в разработке обычно задействовано несколько специалистов – коммуникация между ними упрощается.

Как устроены паттерны

Типичный паттерн — это формализованное решение какой-либо проблемы. Оно может быть представлено как алгоритм или как схема, блоки которой означают части программы.

У паттернов есть свои имена, есть описания, они четко предназначены для решения той или иной проблемы. Имеется и классификация – в первую очередь по тому, для чего нужен тот или иной шаблон.

Отличия паттернов проектирования от архитектурных

Кроме паттернов проектирования, еще есть архитектурные паттерны. Они работают по тому же принципу, но с важным различием. Архитектурные паттерны – высшего уровня, они описывают структуру всего продукта. А паттерны проектирования применяются уже на уровне конкретных объектов, алгоритмов и частей программы.

Если упростить, архитектурный паттерн отвечает на вопрос «как будет устроен продукт». Например, модель MVC — архитектурный паттерн.

А паттерн проектирования отвечает на вопрос «как лучше организовать составные части продукта»: как эффективнее создавать объекты, настраивать обмен данными между ними и их взаимодействие. Паттерны проектирования адаптированы под конкретную задачу, не зависят от языка программирования и не влияют на структуру продукта целиком. Они описывают детали, а не общую архитектуру.

Еще есть идиомы — это тоже формализованные способы решения проблем, но зависящие от языка программирования. Они реализуются на еще более мелком уровне для решения конкретных задач – например, утечки памяти.

Виды паттернов проектирования

Порождающие. Такие шаблоны нужны, чтобы оптимизировать создание того или иного объекта. Порождающие паттерны помогают создавать объекты так, чтобы они эффективно общались с другими, и управлять их работой. Вот несколько примеров:

- «Фабрика» (Factory) — для создания новых объектов придумывают отдельный класс. Он создает объекты как копии некоего эталона;
- «Прототип» (Prototype) — объект сам создает свои копии;
- «Строитель» (Builder) — похож на фабрику, но новые объекты можно модифицировать. Они создаются по сложной логике, а не копируют эталонный;
- «Одиночка» (Singleton) — подразумевает наличие одного большого объекта, который имеет глобальный доступ ко всему;
- «Ленивая инициализация» (Lazy Initialization) — метод, при котором объект инициализируется не сразу, а по мере необходимости.

Существуют и другие шаблоны разной сложности. Для каждой задачи оптимальнее тот или иной паттерн. Конкретное решение зависит от задачи, но в результате должна получиться эффективная и оптимизированная система.

Структурные. Если порождающие паттерны отвечают за создание и взаимодействие объектов, то структурные — за то, как эти объекты структурированы в коде. Они описывают, каким образом простые классы и объекты «собираются» в более сложные.

Вот примеры:

- «Декоратор» (Decorator) — шаблон для подключения дополнительного поведения к объекту;
- «Компоновщик» (Composite) — паттерн, который объединяет несколько объектов в древовидную структуру;
- «Мост» (Bridge) — принцип разделения сущности на абстракцию и реализацию, чтобы теоретическая структура и конкретный объект могли изменяться независимо;
- «Фасад» (Facade) — метод для сведения внешних вызовов к одному объекту;
- «Заместитель» (Proxy) — паттерн, похожий на «Фасад», но со специальным объектом-заместителем, который контролирует доступ к основному.

Это только некоторые примеры. Реальных паттернов намного больше.

Поведенческие. Это паттерны проектирования, которые описывают, как объекты себя ведут и взаимодействуют с другими. Их используют, например, для разделения

обязанностей между разными сущностями или для реагирования на изменения без ошибок.

Примеры поведенческих паттернов:

- «Итератор» (Iterator) — один объект последовательно дает доступ к разным другим, при этом не использует их сложные описания;
- «Наблюдатель» (Observer) – шаблон, при котором объекты узнают об изменениях в других;
- «Хранитель» (Memento) — помогает сохранить объект в каком-то состоянии с возможностью вернуться к нему в будущем;
- «Цепочка ответственности» (Chain of Responsibility) — распределяет ответственность за те или иные задачи на разные объекты;
- «Посредник» (Mediator) — организует слабые связи между объектами, чтобы снизить их зависимость друг от друга.

Как понять, какой паттерн применить

У каждого паттерна своя область использования. Опытные разработчики понимают, где что использовать, по самой специфике задачи, но в начале пути это может быть сложно. Поэтому новичкам советуют разделять выбор паттерна на более мелкие шаги:

- выделить сущности, которые используются в процессе;
- продумать связи между ними;
- абстрагировать получившуюся систему от конкретной задачи;
- посмотреть, не подходит ли проблема по смыслу на что-то, для чего есть паттерн;
- выбрать несколько паттернов из нужной группы и посмотреть какой подходит лучше;
- продумать конкретную реализацию этого паттерна с учетом особенностей задачи.

Это выглядит сложно, но со временем придет привычка. Опытные разработчики уже «набили руку», поэтому проблемы с выбором паттерна у них возникают намного реже. Когда сформируется понимание и наберется практика, будет проще.

Преимущества паттернов проектирования

- Ускоряют и облегчают написание кода.
- Позволяют не «изобретать велосипед», а воспользоваться готовым проверенным принципом.
- При грамотном использовании делают код более читаемым и эффективным.
- Упрощают взаимопонимание между разработчиками.
- Помогают избавиться от типовых ошибок.
- Не зависят от языка программирования и его особенностей.
- Позволяют реализовывать сложные задачи быстрее и проще.

Недостатки паттернов проектирования

- Использование паттернов ради паттернов, наоборот, усложняет код и запутывает разработчиков.
- Неправильное применение того или иного шаблона способно сделать программу менее эффективной.
- Паттерны неуниверсальны: в одной задаче конкретный паттерн подойдет, в другой нет.
- На ранних этапах изучения бывает сложно выбрать подходящий для конкретной проблемы паттерн.
- Из-за сильной связи с объектно-ориентированным программированием использование паттернов в других

парадигмах ограничено. Хотя, например, в функциональном программировании они могут применяться — просто реализуются иначе.

Стоит ли пользоваться паттернами

Наличие недостатков не делает саму идею паттернов плохой. Просто это инструмент, которым нужно пользоваться с умом. Не стоит применять шаблоны там, где можно без них обойтись, просто ради «красоты». Если же использовать их в местах, где они действительно нужны — они станут хорошей помощью в работе программиста.

Более того: на собеседованиях уровня Middle и выше почти всегда спрашивают, знаком ли соискатель с паттернами проектирования. То есть их знание и умение ими пользоваться — практически обязательное условие для программиста уровня выше начального. И неважно, на чем вы пишете: Python, Java, JavaScript или что-нибудь еще.

Как начать работать с паттернами

Паттерны проектирования для новичков — не задача первостепенной важности. К ним обычно переходят люди, у которых есть определенный опыт в программировании, успевшие изучить базовые принципы. Но для перехода на уровень Middle и выше они понадобятся.

Сначала нужно освоить особенности программирования на выбранном языке — некоторые вещи можно почерпнуть уже оттуда. Например, в JS активно используются декораторы. Затем стоит переходить к изучению самих паттернов: какие они бывают, как устроены, что собой представляют.

Освоив теоретическую часть, можно тренироваться. Ведь фактическая реализация порой может серьезно отличаться от теоретического описания, поэтому нужна практика.